# Movie Ticketing System

# Software Requirements Specification

# Version 4.0

# April 9, 2026

Group 1
Elisa Han, Ruhi Singh, Ella Lamie

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Spring 2026

## Revision History

| Date | Description | Author | Comments |
|---|---|---|---|
| 2-15-26 | Version 1 | Elisa Han, Ruhi Singh, Ella Lamie | First Revision |
| 2-18-26 | Version 2 | Elisa Han, Ruhi Singh, Ella Lamie | Edits |
| 3-5-26 | Version 3 | Elisa Han, Ruhi Singh, Ella Lamie | Made the software design specification |
| 4-8-26 | Version 4 | Elisa Han, Ruhi Singh, Ella Lamie | Make the database section (section 6) |

## Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

| Signature | Printed Name | Title | Date |
|---|---|---|---|
| | Ella Lamie | Software Eng. | 3/16/26 |
| | Ruhi Singh | Software Eng. | 3/16/26 |
| | Elisa Han | Software Eng. | 3/16/26 |
| | Dr. Gus Hanna | Instructor, CS 250 | 3/5/26 |
| | | | |

# Table of Contents

**Revision History** ..... ii

**Document Approval** ..... ii

**1. Introduction** ..... 1

1.1 Purpose ..... 1
1.2 Scope ..... 1
1.3 Definitions, Acronyms, and Abbreviations ..... 1
1.4 References ..... 2
1.5 Overview ..... 2

**2. General Description** ..... 2

2.1 Product Perspective ..... 3
2.2 Product Functions ..... 3
2.3 User Characteristics ..... 4
2.4 General Constraints ..... 4
2.5 Assumptions and Dependencies ..... 4

**3. Specific Requirements** ..... 5

3.1 External Interface Requirements ..... 5
*3.1.1 User Interfaces* ..... 5
*3.1.2 Hardware Interfaces* ..... 6
*3.1.3 Software Interfaces* ..... 6
*3.1.4 Communications Interfaces* ..... 6
3.2 Functional Requirements ..... 6
*3.2.1 <Functional Requirement or Feature #1>* ..... 6
*3.2.2 <Functional Requirement or Feature #2>* ..... 7
3.3 Use Cases ..... 8
*3.3.1 Use Case #1* ..... 8
*3.3.2 Use Case #2* ..... 9
*3.3.3 Use Case #3* ..... 9
3.4 Classes / Objects ..... 10
*3.4.1 <Class / Object #1>* ..... 10
*3.4.2 <Class / Object #2>* ..... 10
3.5 Non-Functional Requirements ..... 11
*3.5.1 Performance* ..... 11
*3.5.2 Reliability* ..... 11
*3.5.3 Availability* ..... 11
*3.5.4 Security* ..... 11
*3.5.5 Maintainability* ..... 11
*3.5.6 Portability* ..... 11
3.6 Inverse Requirements ..... 11
3.7 Design Constraints ..... 11
3.8 Logical Requirements ..... 11
3.9 Other Requirements ..... 12

**4. Change Management Process** ..... 12

**5. Design** ..... 12

5.1 System Description ..... 12

5.2 Software Architecture Overview ..... 13
*5.2.1 UML Diagram* ..... 13
*5.2.2 UML Diagram Description* ..... 13
*5.2.3 SWA Diagram* ..... 15
*5.2.4 SWA Diagram Description* ..... 15

5.3 Development Plan and Timeline..... 16
*5.3.1 List of Tasks..... 16*
*5.3.2 Timeline..... 16*
*5.3.3 Task Partitioning and Team Member Responsibilities..... 16*

5.4 UML Design Diagram Modifications..... 16

5.5 Test Plan..... 17

*5.5.1 Test Sets and Test Vectors..... 17*
*5.5.2 How Selected Test Cover Targeted Features..... 20*

5.6 Test Case Samples..... 20

5.7 GitHub Link..... 23

**6. Data Management.....23**

6.1 Data Management Strategy..... 23

6.2 Data Management SQL Diagrams..... 23

*6.2.1 Movie SQL Diagram Example..... 23*
*6,2.2 Showtime SQL Diagram Example..... 24*
*6.2.3 Seats SQL Diagram Example..... 24*
*6.2.4 Tickets and Payment SQL Diagram Example..... 24*
*6.2.5 Customer Info SQL Diagram Example..... 24*

6.3 Entity Relationship Diagram..... 25

6.4 How Many Databases were Chosen..... 25

6.5 How we Split Up the Data Logically..... 26

6.6 Possible Alternatives..... 26

6.7 Tradeoffs Between Our Choice and Alternatives..... 26

6.8 SWA Diagram Changes..... 27

# 1. Introduction

This SRS Document includes information about the Movie Ticketing System. Section 1 includes the purpose, scope, any definitions/acronyms/abbreviations the user needs to know, references, and a general overview. Section 2 contains the general description, the product perspective in relation to other products, the functions the product will perform, the user characteristics, constraints, and assumptions/dependencies. Section 3 provides an overview of requirements such as the external interface requirements, functional requirements, use cases, classes and objects, non-functional requirements, inverse requirements, design constraints, logical database requirements, and any other requirements. Section 4 goes over analysis models, including sequence diagrams, data flow diagrams, and state transition diagrams. Section 5 goes over the change management process. The appendices include any additional information. The system is needed in order to make ticket buying easier for both employees and customers, which should be the client's goal as work hours and customer complications will be to a minimum. The major features of the Movie Ticketing System are movie browsing, showtime listings, and purchasing and payment support.

## 1.1 Purpose

The purpose of this SRS is to outline the requirements, functionality, and constraints of building a theatre ticketing system, enabling a software engineer to design and develop an adequate and efficient web-based software system. Administrators can use this SRS as a guide for how the Movie Ticketing System works.

## 1.2 Scope

The Movie Ticketing System allows users, scaling at least 10,000 people simultaneously, to purchase tickets on a data-driven web browser, maintaining a response time of less than 2 seconds per user request. This browser will display the database of movie showtimes, tickets available, and open seats. Seat availability will update in real time to prevent double-booking. It will be able to generate a ticket ID for every ticket purchase in order to prevent scalping. The Movie Ticketing System will be up for 99.99% of the year, excluding scheduled maintenance periods every two months. It will prevent 98% of ticket scalping. The system will be used by customers and administrators in order to purchase and keep track of movie tickets, respectively.

## 1.3 Definitions, Acronyms, and Abbreviations

- Administrator: An employee that manages movies, showtimes, seating, and customer ticket information.
- Customer: A user that browses movies, selects show times and seating, and makes payments through the system.
- Movie showtimes: A scheduled date and time in which the movie will be taking place. Relating to specific movie and movie theaters.
- Ticket ID: a unique identification generated by the system to represent a purchased ticket.

- HTTPS (HyperText Transfer Protocol Secure): A secure communication protocol used to encrypt data transmitted between the user and the system.

### 1.4 References

Bandakkanavar, R. (2023, May 8). *Software Requirements Specification document with example - Krazytech*. Krazytech. https://krazytech.com/projects/sample-software-requirements-specificationsrs-report-airline-database

Institute Of Electrical And Electronics Engineers. (1998). *IEEE recommended practice for software requirements specifications.* Ieee.

These sources were not used as a reference throughout the SRS, but rather used as a guide on how to properly format the SRS.

### 1.5 Overview

Section 1 provides an introduction to the SRS, including the purpose, the scope, definitions, and structure of the document.

Section 2 gives a general description of the system, including user characteristics, assumptions, constraints, and overall system environment.

Section 3 provides the specific requirements of the system. This is the primary reference for interface design, including user interface, hardware and software. There are detailed functional requirements that map out the detailed features of the system and how the user is to interact with them.

Section 4 presents analysis models such as sequence diagrams, data flow diagrams, and state-transition diagrams that support the requirements defined in Section 3.

Section 5 describes the change management process used to update and maintain this SRS throughout its life cycle

This SRS is organized to move from general system requirements and concepts to detailed and testable features and requirements for clarity and traceability.

## 2. General Description

The Movie Ticketing System is affected by the implementation software, the hardware limitations, and the website it is connected to. An overview of its requirements is provided below, but this section does not state specific requirements.

Requirements:
- Website for implementation
- 24/7 availability
- Safety and security
- Database connection

## 2.1 Product Perspective

The Movie Ticketing System stores the following information:
- Information about ticket buyer
  It includes name, age, and ID for buying tickets.
- Information about movies
  It includes names of movies out, dates released, availability, and theater numbers. This is so the customer can choose what to watch.
- Ticketing ID system
  This is a custom ID for every ticket in order to prevent scalping.

The Movie Ticketing System is not independent and will be contained on each movie theater's website. Refer to Section 3.1 for interface requirements.

## 2.2 Product Functions

The Movie Ticketing System will have various functions:
Note that there is some overlap between the functions of the administrator and the functions of the customer.
From the administrator's side:
- getID will return a person's ID number
- getName will return a person's name
- getAge will return a person's age
  - All of these must be valid in order to proceed with reserveSeat
- addShowtime will let the administrator add a showtime for a movie
- removeShowtime will let the administrator delete a showtime for a movie
- editShowtime will let the administrator edit a showtime for a movie
- checkAvailability will let the administrator check how many seats are available
- editSeatAvailability will let the administrator edit the seat availability for a certain showtime

From the customer's side:
- checkAvailability will let the customer check how many seats are available
- reserveSeat will reserve a seat after having paid
- generateTicket will allow the user to generate their ticket after payment has been made
- deleteReservation will allow a user to cancel their seat reservation and ticket

## 2.3 User Characteristics

Users should be able to get information about movie availability and scheduling. The intended users are both customers and employees.
The customer should be able to:

- View all movies
- Choose a movie and a time
- Choose seating arrangements
- Schedule a movie ticket
- Cancel a movie ticket

The educational level and experience is not needed with customers.
However, the employee has all of the same functions as a user but including management functions:

- See all information about people who have scheduled a ticket
- Be able to edit movies, showtimes, and seat availability

The employee must have:

- At least a high school degree
- Must have experience with databases in order to make sense of the Movie Ticketing System

## 2.4 General Constraints

1. Hardware limitations that involve too many people trying to access the system at once
2. Reliability requirements in that if two people try to buy the last ticket at the exact same time, both might get it
3. Safety and security considerations in that someone might hack the system

## 2.5 Assumptions and Dependencies

- Assuming that the Movie Ticketing System will be used in coordination with a movie theater's website, the Movie Ticketing System will be dependent and change with the type of software the website is implemented on.

# 3. Specific Requirements

This section goes over the specific requirements needed to understand and implement the Movie Ticketing System, including the external interface requirements, the inverse requirements, the logical database requirements, the functional requirements, and the non-functional requirements. It lists each requirement with a priority level and includes various functions. It also includes three use cases and a list of the Movie Ticketing System's classes and objects, as well as design constraints.

## 3.1 External Interface Requirements

### 3.1.1 User Interfaces

**3.1.1.1** High Priority- The Movie Ticketing System can be accessed via:

- A mobile device
- A computer (desktop/laptop)
- A tablet

**3.1.1.2** High Priority- The Movie Ticketing System will show:

- List of available movies
- Showtimes for each available movie
- Seats available per showtimes
- "Movie is full" when user tries to get seats to a full movie

**3.1.1.3** High Priority- What the user can select using their touch input, keyboard, or mouse:

- A showtime for a specific movie
- One or more available seats

` **3.1.1.4** Medium Priority- Confirmation screen before payment:

- Title of movie
- Time of movie
- Seats selected
- Price

### 3.1.2 Hardware Interfaces

**3.1.2.1** High Priority- The Movie Ticketing System will operate on

- A mobile device
- A computer (desktop/laptop)
- A tablet

### 3.1.3 Software Interfaces

**3.1.3.1** High Priority- The Movie Ticketing System will interact with a database to store

- Movies playing
- Showtimes of each movie
- Seats available and taken
- Tickets bought

**3.1.3.2** High Priority- The Movie Ticketing System will partner with a payment processor in order to facilitate payments from customers

### 3.1.4 Communications Interfaces

**3.1.4.1** High Priority- The Movie Ticketing System will use https

- Allows for more safety from the both the user's and the system's side

## 3.2 Functional Requirements

### 3.2.1 Movie Browsing

3.2.1.1 Introduction

- Allows the user to look at all available movies
- Allows the user to look at each of their available showtimes.

3.2.1.2 Inputs

- The database will keep track of the available movies and showtimes and will input that into the system.

- No inputs from the user.

3.2.1.3 Processing

- The Movie Ticketing System will be able to get movies, showtimes, and seat availability from the database.

3.2.1.4 Outputs

- A list of all available movies
- A list of all the corresponding showtimes.

3.2.1.5 Error Handling

- If the input from the database fails, display an error message: “Cannot load movies from database”

**3.2.2 Seat Selection and Payment**

3.2.2.1 Introduction

- Allows the user to select seats
- Allows the user to pay for the seats in advance.

3.2.2.2 Inputs

- Selected movie
- Showtime
- Payment information
- Seat(s).

3.2.2.3 Processing

- The Movie Ticketing System will make sure the seat is available, and reserve the seats for 30 minutes, which, if the user has not paid, will un-reserve the seats.
- If not, it will make sure the payment information is valid, send a request to the payment processor, and confirm the payment and seats.

3.2.2.4 Outputs

- A map of where the seats are in the theater
- A confirmation message saying that the user has paid
- A ticket ID.

3.2.2.5 Error Handling

- If the seat becomes unavailable, display an error message saying “The seat is no longer available. Please pick another seat”.
- If the payment was unsuccessful, display an error message saying “The payment was unsuccessful. Try again”.

**3.2.3 Administration Access**

3.2.3.1 Introduction

- Allows the administrator to access
    - Movies
    - Showtimes
    - Tickets
    - Seats
    - Personal information

3.2.3.2 Inputs

- Movie details
- Showtime details
- Seat available

3.2.3.3 Processing

- The administrator will be able to access all of the information of ticketholders
- May edit movies, showtimes, and seating if they see fit

3.2.3.4 Outputs

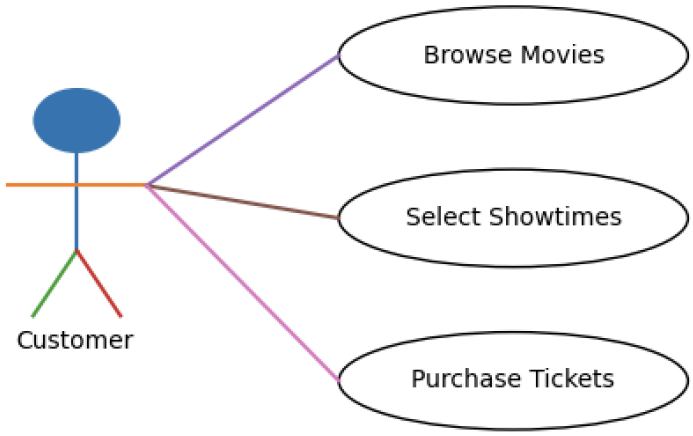
- A confirmation message will be shown after an edit is made to assure the administrator it went through

3.2.3.5 Error Handling

- If the edits are invalid, display an error message saying “Invalid inputs”

## 3.3 Use Cases

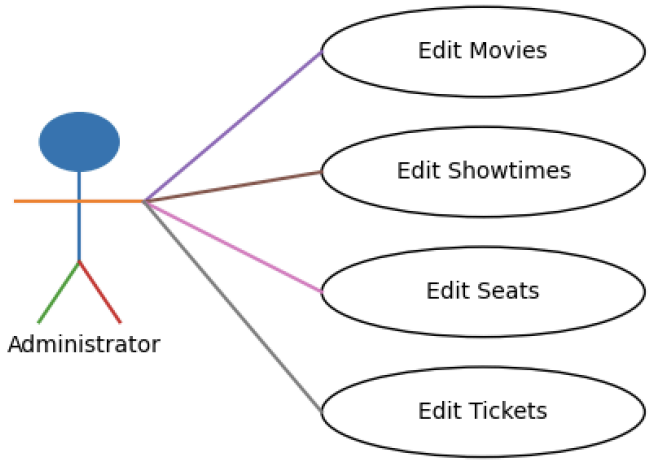
### 3.3.1 Use Case #1: The Customer’s Privileges




Description:

1. Customer can browse through the movies available
2. Customer can select the showtime they want for a particular movie
3. Customer can purchase their tickets

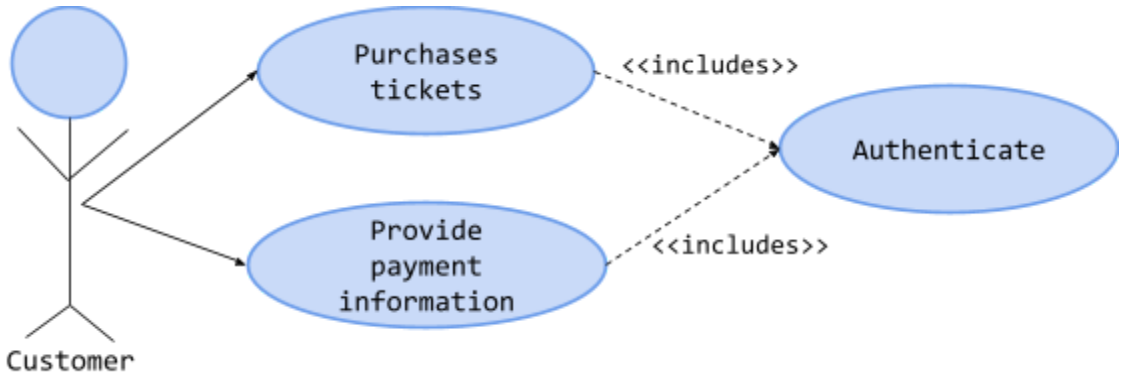
### 3.3.2 Use Case #2: The Administrator Privileges




Description:

1. Administrator can edit movies (i.e. adding or deleting movies)
2. Administrator can edit showtimes for a particular movie
3. Administrator can edit seats for a particular movie
4. Administrator can edit customer tickets

### 3.3.3 Use Case #3: Purchasing Tickets




Description:

1. Customers purchase tickets.
2. Customers provide payment after ticket claiming.
3. The system authenticates the order.

## 3.4 Classes / Objects

**3.4.1 Movie**

3.4.1.1 Attributes

- movieID
- movieName
- movieTime
- movieRating
- movieDescription

3.4.1.2 Functions

- getMovieID()
- getMovieName()
- getMovieTime()
- getMovieDescription()

```
Movie
- movieID : int
- movieName : String
- movieTime : int
- movieRating : double
- movieDescription : String
+ getMovieID() : int
+ getMovieName() : String
+ getMovieTime() : int
+ getMovieDescription() : String
```

References: 3.2.1 and 3.2.3

**3.4.2 Showtimes**

3.4.1.1 Attributes

- showtimeID
- movieID
- startTime

3.4.1.2 Functions

- getShowtimeID()
- getMovieID()
- checkAvailibilty()

```
Showtimes
- showtimeID : int
- movieID : int
- startTime : int
+ getShowtimeID() : int
+ getMovieID() : int
+ hasSeatsAvail() : bool
```

References: 3.2.2 and 3.2.3

**3.4.3 Seat**

3.4.1.1 Attributes

- seatID
- availability
- priceSeat
- reserveSeatSuccess
- deleteReservationSuccess

3.4.1.2 Functions

- reserveSeat()
- checkAvailability()
- deleteReservation()
- getSeatID()

```
Seat
- seatID : int
- availability : bool
- priceSeat : int
- reserveSeatSuccess : bool
- deleteReservationSuccess : bool
+ reserveSeat() : bool
+ checkAvailibily() : bool
+ deleteReservation() : bool
+ getSeatID() : int
```

References: 3.2.2 and 3.2.3

**3.4.4 Admin**

3.4.1.1 Attributes
- adminID
- editMovieSuccess
- editShowtimesSuccess
- editSeatSuccess

3.4.1.2 Functions
- editMovie()
- editShowtimes()
- getCustomerInfo()

```
Admin
- adminID : int
- editMovieSuccess : bool
- editShowtimesSuccess : bool
- editSeatsSuccess : bool
+ editMovie() : void
+ editShowtimes() : void
+ editSeats() : void
+ getCustomerInfo : String
```

**3.4.5 Ticket**

3.4.1.1 Attributes
- ticketID
- priceTicket
- purchaseTicketSuccess

3.4.1.2 Functions
- purchaseTicket()
- getTicketID()
- getPrice()

```
Ticket
- ticketID : int
- priceTicket : double
- purchaseTicketSuccess : bool
+ purchaseTicket() : bool
+ getTicketID() : int
+ getPrice() : double
```

**3.4.6 Payment**

3.4.1.1 Attributes
- typePayment
- paymentSuccess
- paymentID

3.4.1.2 Functions
- makePayment()
- getPaymentID()
- getTypePayment()

```
Payment
- typePayment : String
- paymentSuccess : bool
- paymentID : int
+ makePayment() : bool
+ getPaymentID() : int
+ getTypePayment() : String
```

**3.4.7 CustomerInfo**

3.4.1.1 Attributes
- customerName
- customerAge
- customerID

3.4.1.2 Functions
- getName()
- getAge()
- getID()

```
CustomerInfo
- customerName : String
- customerAge : int
- customerID : int
+ getName() : String
+ getAge() : int
+ getID() : int
```

## 3.5 Non-Functional Requirements

### 3.5.1 Performance

- 95% of user transactions shall be processed in less than a two seconds
- The system shall support 1,000 users at a time

### 3.5.2 Reliability

- System downtime shall not exceed 2 minutes per day
- There shall not be a system failure more than once each 30 days

### 3.5.3 Availability

- Movie Ticketing System shall be up 99.99% of the time per year

### 3.5.4 Security

- Passwords and user IDs will be encrypted
- Payment information will be encrypted

### 3.5.5 Maintainability

- The Movie Ticketing System will update every 2 months
- It will not take more than 30 seconds

### 3.5.6 Portability

- The Movie Ticketing System will be able to be used on Windows, Macs, and Linux

## 3.6 Inverse Requirements

- System must support and operate a web browser to go to the website URL
- System must use HTTPS
- System must consolidate with a payment processor

## 3.7 Design Constraints

- System must be able to handle a minimum of 1000 users at the same time
- System must support user requests within 2 seconds
- The availability of theatre seats must update instantaneously to avoid double-booking
- User information such as emails, passwords, and payment d must be encrypted
- Administrator access must be restricted from public access with secure authentication

## 3.8 Logical Database Requirements

- Customer Database
    - Customer ID
    - Payment ID
    - Payment status
    - Customer’s email

    - Customer's name
    - Customer's age
    - Customer's password (encrypted)
- Movie Database
    - Movie ID
    - Movie showtime ID
    - Movie seat ID
    - Movie ticket ID
    - Movie title
    - Movie date/time
    - Movie seat capacity
    - Movie seat availability
    - Movie seat number
    - Movie price

### 3.9 Other Requirements

- System needs to scale 10,000 users concurrently
- System needs to perform load balancing to not overwhelm servers
- System needs to log transactions and transaction status
- System must display clear error messages
- System must perform efficient error handling
- System must confirm and record successful transactions

## 4. Change Management Process

*Change Request*

- Present a detailed description of the change upon proposal followed by the reason, beneficial impact, and timeline.

*Change Review*

- The team will review the functional and technical requirements of this change as well as the practicality and risks associated with it.

*Initiate Change*

- If all members of the team agree upon the change after the review, the change will be initiated and implemented.

## 5. Design

### 5.1 System Description

The Movie Ticketing System is a software application that is attached to a movie theater company's website. It allows the user, either an administrator or a customer, to access movies

being played at a certain location, movie showtimes, and seats available (including a map of the seat layout) for a certain movie. It allows customers to pay for a movie ticket. The Movie Ticketing System will authenticate payments and provide a unique movie ticket ID. It allows administrators to edit movies, movie showtimes, and seats available, as well as access customer information. In other words, the administrator can override and edit the Movie Ticketing System.

## 5.2 Software Architecture Overview

### 5.2.1 UML Diagram

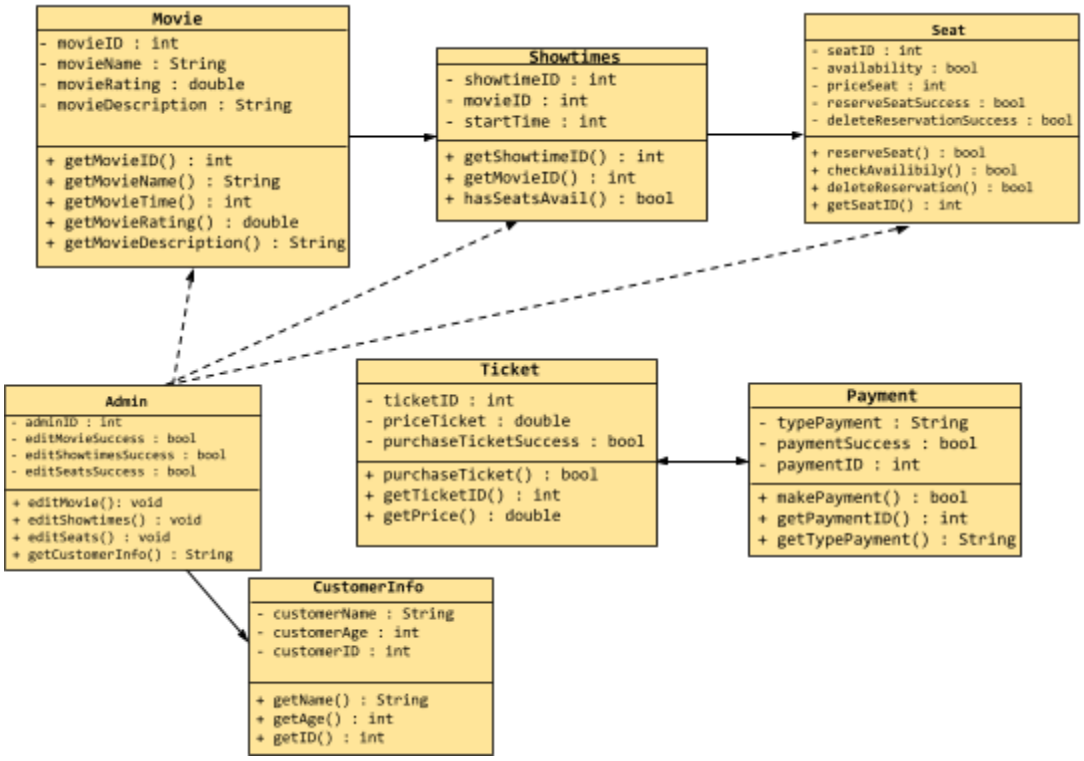



Figure 1

### 5.2.2 UML Diagram Description

Description:

1. Movie Setup
    a. Admin creates / edit movies
    b. Each movie has one or more showtimes
    c. Can get attributes of movies such as the ID (int), name (String), time (String), rating (double), and description (String)
    d. Connected to showtimes and admin as admin has access to movie and showtime has access to movie
    e. No parameters
2. Showtime Selection
    a. A customer selects a movie
    b. The system retrieves related showtimes

    c. Can get attributes such as the ID of the showtime (int), the corresponding movie (int), and seats available (int). The start time (String) is also available
    d. Connected to movie as it can access movie ID, admin as admin can edit showtimes, and seats as seats has access to showtimes
    e. checkAvailability (void) will go into seats and return how many are available for this showtime
    f. No parameters
3. Seat Reservation
    a. For a selected showtime:
        i. Available seats are displayed
        ii. Customer selects seats
        iii. reserveSeat() updates to show that the seat is no longer available
    b. Can get the seat ID (int), availability (int), and the price of a seat (int)
    c. Customer can also delete their seat reservation (void) which frees up the reserved seat and gives the customer back their money if the cancellation is within 30 mins of the movie starting
    d. checkAvailability (void) comes from showtimes which shows a map of which seats are available
    e. Connected to showtimes as it has access to showtimes and to admin because admin has access to editing seats
    f. No parameters
4. Ticket Creation
    a. After seat reservation:
        i. A ticket is created
        ii. Ticket stores price
    b. Connected to payment as ticket stores price and allows purchase
    c. Can also get the ticket ID (int) and price (int) which simply return each
    d. purchaseTicket (void) is called after reserveSeat and takes the user to the page where they enter their information and payment method, and then finally pay
    e. No parameters
5. Payment
    a. Ticket is linked to Payment
        i. makePayment() (void) processes transaction
        ii. if paymentSuccess() (boolean) true:
        iii. Ticket purchase is confirmed
    b. getTypePayment (String) will return the type of payment the user is using (credit card, banking information, cash)
    c. getPaymentID (int) will return the ID of the purchase, which is different than the ticket ID
    d. No parameters
6. Admin
    a. Admin are able to:
        i. Edit movies (void)
        ii. Modify showtimes (void)

iii. Adjust seats (void)
iv. Access customer information (String)

b. Linked to movie, showtimes, customer information, and seats as admin has access to all four
c. Also contains the adminID (int) which is stored to show which admin made what edits/changes
d. No parameters

7. Customer Information
   a. Simply stores the name (String), age (int), and ID (int) of the customer
   b. Contains getter methods so the admin can access the customer information
   c. Connected to admin as admin can access customer information
   d. No parameters

### 5.2.3 SWA Diagram

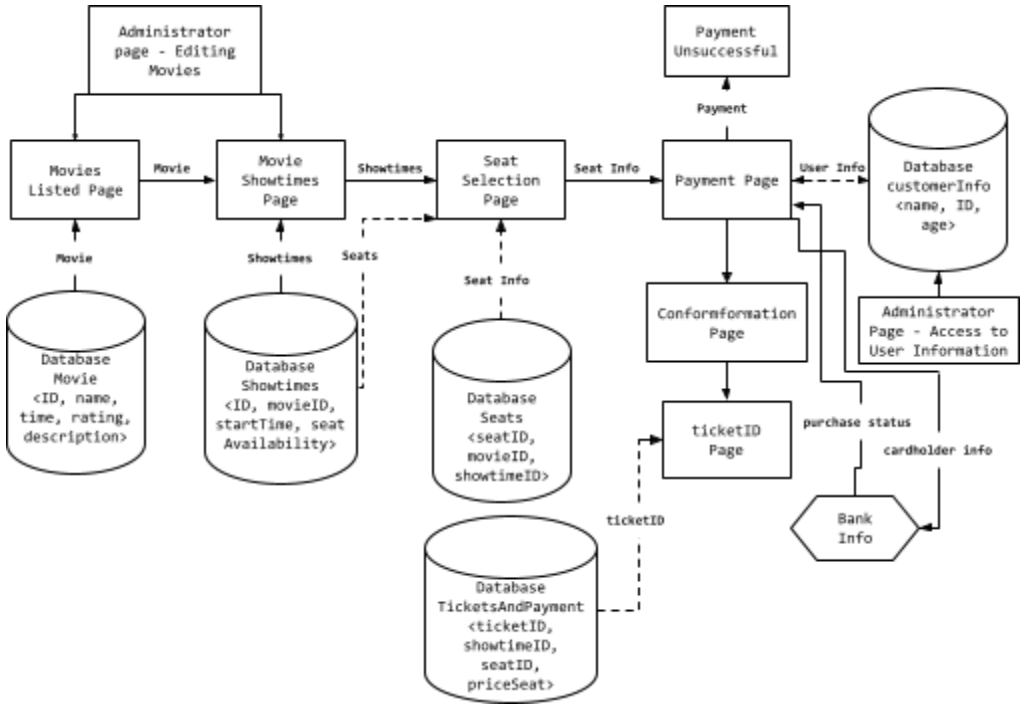



Figure 2

### 5.2.4 SWA Diagram description

First, there is the movies listed page, which takes from the movie database. The administrator page is connected to the movies page and the showtimes page. The showtimes page is connected to the showtimes database. The administrator page is connected to the showtimes page and movies page because the admin can edit both. The showtimes database is also connected to the seat selection page because it also contains information about the seats and availability for each showtime. The seat selection page also takes from the tickets and payment database as it contains information about the payment of and purchasing of the seats. The seat selection page will then take you to the payment page, which also is connected to the customer information database. The

customer information database is also connected to admin because they have access to all the customer information. The payment page will either go to payment unsuccessful or a confirmation page. The purchase status part of the payment page comes from the bank info, and the cardholder info comes from the payment page. The confirmation page goes to the ticketID page because the confirmation page will show a ticketID. The ticketID page also takes from the database tickets and payment as that stores information about tickets and available ticket IDs.

## 5.3 Development Plan and Timeline

### 5.3.1 List of Tasks

1. Implement a fully tested movie class with methods
2. Implement a fully tested showtime class with methods
3. Implement a fully tested seats class with methods
4. Implement a fully tested admin privileges class
5. Implement movie database
6. Implement customer information database
7. Implement a fully tested customer information class with methods
8. Implement a fully tested payment class with methods
9. Implement a fully tested ticket class with methods
10. Connect movie theater website to Movie Ticketing System
11. Implement connection with payment system
12. Do a final run through to make sure there are no bugs

### 5.3.2 Timeline

- March- Finish movie, showtime, seats, and admin class and have them fully tested
- April- Implement the movie and customer information databases, and finish the customer information, payment, and ticket classes (fully tested)
- May- Connect the movie theater website to Movie Ticketing System, connect to a payment system, and run through to make sure there are no bugs

### 5.3.3 Task Partitioning and Team Member Responsibilities

- Ella:
    - March- Implement movie class and have it fully tested
    - April- Implement movie database and customer information class (fully tested)
    - May- Connect the movie theater website to the Movie Ticketing System
- Ruhi:
    - March- Implement showtime class and have it fully tested
    - April- Implement customer information database and payment class (fully tested)
    - Connect to a payment system
- Elisa:
    - March- Implement seats and admin classes and have them fully tested
    - April- Implement ticket class and have it fully tested
    - Do a final run through to make sure there are no bugs

## 5.4 UML Design Diagram Modifications

A few variables were added to the classes, and therefore to the UML diagram in order to measure whether an action (i.e. a method that returns void) was successful or not.

- Seats Class
    - Added bool reserveSeatSuccess
    - Added bool deleteReservationSuccess
- Admin Class
    - Added bool editMovieSuccess
    - Added bool editShowtimesSuccess
    - Added bool editSeatsSuccess
- Ticket Class
    - Added bool purchaseTicketSucces

## 5.5 Test Plan

### 5.5.1 Test Sets and Test Vectors

- Unit Test Sets
    - Test Movie class
        - Test Vectors:
            - {Input = getMovieID, Output = movieID}
            - {Input = getMovieName, Output = movieName}
            - {Input = getMovieTime, Output = movieTime}
            - {Input = getMovieRating, Output = movieRating}
            - {Input = getMovieDescription, Output = movieDescription}
    - Test Showtimes Class
        - Test Vectors:
            - {Input = getMovieID, Output = movieID}
            - {Input = getShowtimeID, Output = showtimeID}
            - {Input = checkAvailability, Output = availability}
    - Test Seat class
        - Test Vectors:
            - {Input = reserveSeat, Output = reserveSeatSuccess}
            - {Input = checkAvailability, Output = availability}
            - {Input = deleteReservation, Output = deleteReservationSuccess}
            - {Input = getSeatID, Output = seatID}
    - Test Admin class
        - Test Vectors:
            - {Input = editMovie, Output = editMovieSuccess}
            - {Input = editShowtimes, Output = editShowtimesSuccess}
            - {Input = editSeats, Output = editSeatsSuccess}
            - {Input = getCustomerInfo, Output = customerName, customerAge, customerID}
    - Test Ticket class
        - Test Vectors:
            - {Input = purchaseTicket, Output = purchaseTicketSuccess}
            - {Input = getTicketID, Output = ticketID}
            - {Input = getPrice, Output = priceTicket}
    - Test Payment class

        - Test Vectors:
            - {Input = makePayment, Output = paymentSuccess}
            - {Input = getPaymentID, Output = paymentID}
            - {Input = getTypePayment, Output = typePayment}
    - Test CustomerInfo class
        - Test Vectors:
            - {Input = getName, Output = customerName}
            - {Input = getAge, Output = customerAge}
            - {Input = getID, Output = customerID}
- Integration Test Sets
    - Test integration between movie theater website and Movie Ticketing System
        - Test Vectors:
            - {Input = user clicks on Movie Ticketing System button on movie theater website, Output = Movie Ticketing System opens}
    - Test integration between Movie database and Movies listed page
        - Test Vectors:
            - {Input = getMovieID, Output = movieID}
            - {Input = getMovieName, Output = movieName}
            - {Input = getMovieTime, Output = movieTime}
            - {Input = getMovieRating, Output = movieRating}
            - {Input = getMovieDescription, Output = movieDescription}
            - {Input = editMovie, Output = editMovieSuccess}
    - Test integration between Showtimes database and movie showtimes page
        - Test Vectors:
            - {Input = getMovieID, Output = movieID}
            - {Input = getShowtimeID, Output = showtimeID}
            - {Input = checkAvailability, Output = availability}
            - {Input = editShowtimes, Output = editShowtimesSuccess}
    - Test integration between Showtimes database and seats page
        - Test Vectors:
            - {Input = reserveSeat, Output = reserveSeatSuccess}
            - {Input = checkAvailability, Output = availability}
            - {Input = deleteReservation, Output = deleteReservationSuccess}
            - {Input = getSeatID, Output = seatID}
            - {Input = editSeats, Output = editSeatsSuccess}
    - Test integration between Tickets and Payment database and seats page
        - Test Vectors:
            - {Input = reserveSeat, Output = reserveSeatSuccess}
            - {Input = deleteReservation, Output = deleteReservationSuccess}
            - {Input = getSeatID, Output = seatID}
    - Test integration between Tickets and Payment database and ticket ID page
        - Test Vectors:
            - {Input = purchaseTicket, Output = purchaseTicketSuccess}

            - {Input = getTicketID, Output = ticketID}
            - {Input = getPrice, Output = priceTicket}
            - {Input = deleteReservation, Output = deleteReservationSuccess}
            - {Input = getSeatID, Output = seatID}
            - {Input = getMovieID, Output = movieID}
            - {Input = getShowtimeID, Output = showtimeID}
    - Test integration between Customer Info database and payment page
        - Test Vectors:
            - {Input = getName, Output = customerName}
            - {Input = getAge, Output = customerAge}
            - {Input = getID, Output = customerID}
    - Test integration between Customer Info database and admin page:
        - Test Vectors:
            - {Input = getName, Output = customerName}
            - {Input = getAge, Output = customerAge}
            - {Input = getID, Output = customerID}
- System Test Set
    - Test that user can navigate to Movie Ticketing System from movie theater website, can select a movie, showtime, and seat, put in their information and pay, and get a movie ticket
        - Test Vectors:
            - {Input = user clicks on Movie Ticketing System button on movie theater website, Output = Movie Ticketing System opens}
            - {Input = user selects a movie, Output = Movie Ticketing System opens Movie Showtimes page}
            - {Input = user clicks on a showtime, Output = Movie Ticketing System opens Seat Selection page}
            - {Input = user selects a seat, Output = Movie Ticketing System opens Payment page}
            - {Input = user's payment is unsuccessful, Output = Movie Ticketing System opens Payment Unsuccessful page}
            - {Input = user's payment is successful, Output = Movie Ticketing System opens Confirmation and Ticket ID pages}
    - Test that admin can navigate to the Movie Ticketing System from the movie theater website, edit movies, seats, and showtimes, and can access customer information
        - Test Vectors:
            - {Input = admin clicks on Movie Ticketing System button on movie theater website, Output = Movie Ticketing System opens}
            - {Input = admin navigates to Administrator Page: Edit Movies, Output = admin is able to edit movies and showtimes}

- {Input = admin navigates to Administrator Page: Access to Customer Information, Output = admin is able to view all customer's names, ages, and IDs}

### 5.5.2 How Selected Tests Cover Targeted Features

The unit tests confirm that all of our individual major classes carry out the correct functions without error.

- The Showtime class tests make sure that things such as the seat availability and the showtime data are all operating well.
- The Movie class tests verify that all of the information for the movie class such as the name, ID, time, descriptions, and rating can all be accessed efficiently and effectively.
- The Seats class tests ensure that things such as the reservation and seat information are all correctly managed
- The CustomerInfo, Payment, Ticket, and Admin class tests check and confirm that the payment processing, tickets, administrative access and editing, and customer data all perform and are maintained correctly in the system.

The integration tests ensure that all of the individual class components come together and function properly cooperatively

- The Movie class & Movie database tests make sure that the system correctly displays the right movie data to the user, matching its database content
- The Showtime database and the seat selection page in our system are tested to ensure that the showtime and seat data are aligned correctly
- The tests between the Tickets and Payment databases verify that the data regarding the ticket purchasing such as the payment status and ticket ID creation all flows correctly
- The Customer Information database, payment pages, and the admin access tests confirm that the customer's data is easily accessible when necessary

The system tests make sure all components of the movie ticketing system are working effectively and efficiently as a cohesive whole

- Regarding the customers side, the customer system tests make sure that the purchasing side of the system works effectively. This entails that the user is adequately able to go into the website, choose a movie, the respective showtime, and a seat, and enter their payment in order to be sent to either an error page or a confirmation page that generated a ticket ID.
- Regarding the administrative side, the admin system tests ensure that an administrator is able to login appropriately and perform any administrative functions necessary such as editing the showtimes, movies, seats, or accessing the customer's information.

## 5.6 Test Case Samples

Note that priorities are P0, P1, and P2 where P0 is the highest priority and P2 is the lowest priority.

| Test Cases | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **TestCaseId** | **Component** | **Priority** | **Description/Test Summary** | **Pre-requisites** | **Test Steps** | **Expected Result** | **Actual Result** | **Status** | **Test Executed By** |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| System_Website_01 | Movie Theater Website | P0 | Verify that when a user clicks on the button that says "Movie Ticketing System", they are directed to the Movie Ticketing System homepage (the movie homepage) | Browser is launched and user is on movie theater website | 1. Navigate to the movie theater website 2. Click on button that reads "Movie Ticketing System" using a mouse or touch screen | Movie Ticketing System homepage (movies homepage) should be displayed | Movie Ticketing System homepage is displayed | Pass | TesterE1 |
| Unit_Movie_01 | Movie class | P0 | Verify when a user clicks on a movie, they are immediately directed to the showtimes for that movie | User is already on the movie homepage | 1. Scroll through movies 2. Click on a movie 3. Click back to return to movie homepage | User should be able to scroll through movies, click on a movie and see the available showtimes for that movie, and use the back button to return to the movie homepage | User was able to scroll through movies, select a movie and see the display of available showtimes, and use the back arrow to get back to the homepage | Pass | TesterR |
| Integration_Database_01 | Movie class and Movie database | P0 | Verify that the movies listed in the movie homepage match all of those in the movie database including ID number, name, rating, and description | User is on the movie homepage | 1. Navigate to database 2. Navigate to movie homepage 3. Make sure each movie is accounted for and matches the description in the database | The database and movie homepage should match | The database and the movie homepage match | Pass | TesterE2 |
| Unit_Showtimes_01 | Showtimes class | P0 | Verify that when a user clicks on a specific showtimes for a previously selected movie, that the user will be directed to the seat selection page | User is on the showtimes page | 1. Scroll through showtimes 2. Click on a showtime | The user should be directed to the seat selection page | The user was directed to the seat selection page | Pass | TesterE1 |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Integration_Database_02 | Showtimes class and Showtimes database | P0 | Verify that the showtimes listed in the showtimes homepage match all of those in the showtimes database including ID number, start time, and seat availability | User is on the showtimes page | 1. Navigate to database 2. Navigate to showtimes homepage 3. Make sure each showtimes is accounted for and matches the description in the database | The database and showtimes homepage should match | The database and showtimes homepage matched | Pass | TesterR |
| Unit_Seats_01 | Seats class | P0 | Verify that a user can look at and click on seats in the seats page, and when they click on a seat they are directed to the payment page | User is on seats page | 1. Scroll through the seats 2. Click on a seat | The user should be able to click on a seat and is directed to the payment page | The user was able to click on a seat and was able to be directed to the payment page | Pass | TesterE2 |
| System_Website_02 | Movie Theater Website | P1 | Verify when an admin of the movie theater website is on the movie theater website, if they click on the button to the movie ticketing system, they will be navigated to the admin version of the Movie Ticketing System website | Browser is launched and user is on movie theater website | 1. Navigate to the movie theater website while logged into the admin account 2. Click on the button that reads “Movie Ticketing System” using a mouse or touch screen | The admin should be directed to the admin page of the Movie Ticketing System | The admin was directed to the admin page of the Movie Ticketing System | Pass | TesterE1 |
| Unit_Payment_01 | Payment class | P0 | Verify the payment page where the user will be able to put in their information and connect to their credit card | User is on payment page | 1. Navigate to payment page 2. Enter credit card | The user should be able to enter information in and be directed to a payment successful page | The user was able to enter information and was directed to a payment successful page | Pass | TesterR |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Unit_Payment_02 | Payment class and Bank Info database | P2 | Verify the user is able to have an option to enter their bank information in the payment page instead of their credit card | User is on payment page | 1. Navigate to payment page<br>2. Enter bank information | The user should be able to enter bank information successfully and will be navigated to payment successful page | The user put in their bank information and was navigated to a payment successful page | Pass | Tester E2 |
| Unit_Payment_03 | Payment class | P2 | Verify the user will be directed to a page unsuccessful page if any payment information was not correct or incomplete | User is on payment page | 1. Navigate to payment page<br>2. Put in incorrect and incomplete information | The user should be directed to the payment unsuccessful page | The user was directed to the payment unsuccessful page | Pass | TesterE1 |

### 5.7 GitHub Link

Below find the link to the GitHub:
https://github.com/EllaLamie/Software_Design_Specification.git

## 6. Data Management

### 6.1 Data Management Strategy

We will be using SQL because we have structured data that we are using in the Movie Ticketing System. For example, we have movies, showtimes, seats, and tickets, among other things. Also, there are relationships that exist between different parts of the Movie Ticketing System which relates to the relational part of the SQL data management strategy. Consistency is also something needed for the Movie Ticketing System, and there will not be any large changes to the amount of data, so the SQL is a better fit. In our case, the ACID properties are needed over the CAP properties for the NoSQL. Specifically, we are thinking of using MySQL because, in researching, it seemed to be the best fit. It is very scalable, supports relationships between different variables, is reliable, and also is supported on Windows, Linux, and Mac. The diagrams below give a visual representation of our strategy.

### 6.2 Data Management SQL Diagrams

#### 6.2.1 Movie SQL Diagram Example

| ID | Name | Rating | Time | Description |
|---|---|---|---|---|
| 0000 | “Cinderella” | G | 90 | “Glass slipper” |

| | | | | |
|---|---|---|---|---|
| 0001 | “Snow White” | PG | 100 | “Poison apple” |
| 0002 | “Tangled” | G | 95 | “Long hair” |

### 6.2.1 Showtime SQL Diagram Example

| Showtime ID | Movie ID | Start time | Seats Available |
|---|---|---|---|
| 1000 | 0000 | 11:00 | 20 |
| 1001 | 0001 | 14:00 | 9 |
| 1002 | 0002 | 13:00 | 12 |

### 6.2.2 Seats SQL Diagram Example

| Seat ID | Movie ID | Showtime ID |
|---|---|---|
| 20000 | 0000 | 1000 |
| 20001 | 0001 | 1001 |
| 20002 | 0002 | 1002 |

### 6.2.2 Tickets and Payment SQL Diagram Example

| Ticket ID | Showtime ID | Seat ID | Price of Seat |
|---|---|---|---|
| 00000000 | 1000 | 20000 | 15 |
| 00000001 | 1001 | 20001 | 10 |
| 00000002 | 1002 | 20002 | 9 |

### 6.2.3 Customer Info SQL Diagram Example

| ID | Name | Age |
|---|---|---|
| 00 | “Bob” | 70 |
| 01 | “Steve” | 20 |

| | | |
|---|---|---|
| 02 | “Elizabeth” | 50 |

## 6.3 Entity Relationship Diagram

<u>Data Dictionary</u>
Movie id: unique id of the current movie
Name: name of the current movie
Rating: rating of the current movie
Time: duration of current movie
Description: description of current movie
Showtime id: unique id of a specific showtime
Start time: time when the movie starts for a specific showtime
Seats available: seats available for a specific showtime
Seat id: seat id of a certain seat
Ticket id: id of a ticket to the movies
Seat price: price of a certain seat
Id: customer’s id
Name: customer’s name
Age: customer’s age

## 6.4 How Many Databases were Chosen

We chose to do five databases in order to be able to have the databases be as organized as possible and so that all of the different pages will not be taking from the same database. We

discuss why we chose doing different databases rather than making a single database or an event driven database in future sections (section 6.7).

## 6.5 How We Split Up the Data Logically

We decided to split up the data into five different databases where each corresponds to a main page on the Movie Ticketing System website. Therefore, each page will pull from a specific database: the movie page will pull from the movie database, the showtimes page will pull from the showtimes database, the seats page will pull from the seats database, the tickets and payment page will pull from the tickets and payment database, and the admin page will pull from the customer info database.

## 6.6 Possible Alternatives

The main possible alternative we could use is some NoSQL database. This means it would be non-relational or distributed, either based on documents, key-value pairs, graphs, or columns, cheaper to scale, and better when storing larger sets of data. In our research, we came across MongoDB which would be the best NoSQL option for us. It is in the document category of NoSQL databases and stores data in documents. The reason it would be good for us is that it has horizontal scalability which would be beneficial for many users, it is in the document category, which matches how our movies, showtimes, and seats are hierarchical, and it is flexible. Another possible alternative would be Redis which is another NoSQL database, but in the key-value category. It is fast and real-time which would be very helpful for people choosing movies, showtimes, and seats. Another alternative would be another SQL database, PostgreSQL. It is very similar to MySQL but just another SQL we could try. As for organization of data, we also could have used a single database rather than five different databases or we could have used an event database where everything was organized into an event that included the ticket which included the movie, showtime, and seat.

## 6.7 Tradeoffs Between Our Choice and Alternatives

Technology:

- Between MySQL and MongoDB:
    - MongoDB, because it is NoSQL and in the document category, is very difficult to link or connect different documents, which could be difficult when someone wants to reflect the showtimes based on the movies, and the seats based on the showtimes in our case where we have multiple databases. However, MySQL cannot be scaled horizontally which would be beneficial to many users, while MongoDB can. All in all though, because of the linking we need and the scalability, MySQL turns out to be the better choice.
- Between MySQL and Redis:
    - Redis is very fast, while MySQL is less fast, but Redis does not have as much space to work with as MySQL. All in all, it seems that MySQL is better because while Redis is fast, it also has more expensive storage and for a lot of data like the Movie Ticketing System has, this will not be ideal. Because Redis is more expensive and given the amount of data we have, MySQL becomes the better choice.

- Between MySQL and PostgreSQL:
    - PostgreSQL is a bit more complicated and complex than MySQL. Even though they are relatively similar, MySQL ends up being the better option because of relative simplicity. However, PostgreSQL is usually better for more complex systems. The Movie Ticketing System is a relatively simple system, so MySQL ends up being the better choice.
- Between SQL and NoSQL:
    - NoSQL is good at handling a lot of data, is very scalable, and has good availability, three things that are very important for a Movie Ticketing System. However, it cannot guarantee ACID properties, it does not have run-time flexibility, and is not consistent. SQL is very consistent and can handle changes in the volume of data and complex queries. However, it is not hierarchical which is how the different sections of the databases are.

Organization of Data:

- Single Database
    - The advantage of having a single database is that everything would pull from it and we would not have to keep track of many different databases. However, the database could get a bit messy and unorganized given that it is storing movie information, showtime information, seating information, ticketing information, and payment information. This is why multiple databases would be a better fit.
- Store Everything in an Event Database
    - The advantage of having an event database is that each ticket would have corresponding movie, showtime, and seat so that would count as one event. This would make it easier for ticket generation but a little less organized and separate than just having individual databases. This option is a bit more complicated than just having separate databases, so having separate databases would be the best option.

## 6.8 SWA Diagram Changes

The change we made to our SWA Diagram to reflect the current database management system was creating a new seats database to store the ID of a specific seat in a specific movie theater, the ID of the movie the seat corresponds to, and the ID for the showtime the movie corresponds to. Because originally the tickets and payment database was what the seat selection page pulled from, we now have that specifically attached to the tickets and payment information, rather than the seats as well. Now the seats have their own separate database. This makes our database system more organized and separated.